

Especificación de Requerimientos

Iteración 4

Ingeniería de Software

Versión del Documento: 5.0.0

Elaborado por:

Fuentes Anica Cesar Candelario

Morales Sánchez Jennifer Lillian

Reyes Colín Luis Manuel

Solares Ramos Luis Mario

Valente Trinidad Miguel Angel

08 / junio / 2026

Control de Versiones

Fase de Desarrollo	Versión del Sistema
Iteración 1	1.0.0
Iteración 2	2.0.0
Iteración 3	3.0.0
Iteración 4	4.0.0
Iteración 5	5.0.0

Índice

Control de Versiones	1
1. Introducción y Alcance	3
1.1. Introducción	3
1.2. Propósito	3
1.3. Alcance del sistema	3
1.4. Limitaciones actuales	4
2. Arquitectura del Sistema	4
2.1. Arquitectura en capas	4
2.2. Seguridad y Autenticación	5
2.3. Flujo Request-Response	5
2.4. DTOs, Domain Objects y Entities	6
2.5. Integración de servicios externos	6
3. Especificación de Requerimientos	6
3.1. Requerimientos Funcionales	6
3.2. Requerimientos No Funcionales	9
4. Casos de Uso	10
5. Diseño de Datos	28
5.1. Modelo Conceptual de Datos	28
5.2. Schema	28

6. Sección Técnica	30
6.1. Tecnologías Utilizadas	30
6.2. Flujo de Autenticación	31
7. Actores de sistema	31
8. Definiciones relevantes	32

1 Introducción y Alcance

1.1 Introducción

El presente documento describe los requerimientos y arquitectura del sistema en su versión 4,0, correspondiente a la cuarta y última iteración de desarrollo. En esta etapa de consolidación, el sistema evoluciona mediante la optimización de la interfaz de usuario, así la integración del mapa en donde se encuentra la mascota a adoptar y la incorporación de una verificación del correo en el momento del registro y se implementan funcionalidades como la eliminación imágenes.

1.2 Propósito

El documento tiene como propósitos principales:

- Especificar los requerimientos finales del sistema, definiendo los nuevos casos de uso funcionales.
- Documentar la consolidación de la arquitectura en capas del backend, el modelo relacional de datos y la intergración con el frontend.
- Traducir reglas de negocio implementadas

1.3 Alcance del sistema

- Respecto al backend: El alcance actual del sistema en esta versión se divide en los módulos que se describirán a continuación, estos operan en un esquema cliente-servidor (React y Spring Boot):
 - Módulo de Gestión de Usuarios:
 - Registro de nuevos usuarios
 - Autenticación segura y generación de sesión basada en tokens (JWT)
 - Consulta y actualización parcial de la información del perfil de usuario
 - Verificación de correo electrónico durante el proceso de registro
 - Módulo de Publicaciones:
 - Creación de anuncios de mascotas asociadas al usuario en sesión
 - Listado general de publicaciones activas con un soporte para filtros como tipo de mascota (dog/cat), código postal, raza
 - Visualización detallada, edición parcial y eliminación de publicaciones del mismo usuario.
 - Integración con API externa de geolocalización para la obtención de las coordenadas a partir del código postal escrito.
 - Módulo de Interacción y Notificaciones:
 - Funcionalidad para que un usuario marque su interés en una publicación ajena

- Consulta de la lista de usuarios interesados al alcance del dueño de la publicación
 - Integración con servicio de correo electrónico (SMTP) para alertar al propietario cuando un usuario registra interés en su mascota.
- Respecto al frontend: La interfaz de usuario, desarrollada en React, comprende una arquitectura de componentes modulares y vistas SPA (Single Page Application) completamente integradas con los servicios y endpoints reales del backend, distribuidas en los siguientes módulos funcionales:
 - Registro de usuario
 - Inicio de sesión
 - Home: Vista principal
 - Visualización de la información del usuario posterior a iniciar sesión
 - Visualización de la información del usuario autenticado
 - Mapa de ubicación general
 - Cierre de sesión

1.4 Limitaciones actuales

En esta Iteración 4, el sistema está limitado a lo siguiente:

- El sistema solo permite registrar únicamente a perros y gatos.

2 Arquitectura del Sistema

El sistema está compuesto por dos componentes principales, el backend y el frontend, los cuales se comunican entre sí, se detallan a continuación:

- **Frontend:** Interfaz de usuario que usa la API, realizando las peticiones HTTP al backend usando API Fetch de Javascript.
- **Backend:** API REST que procesa las solicitudes, se organiza usando el patrón MVC, dividiendo en tres capas:

2.1 Arquitectura en capas

El sistema implementa una arquitectura en capas para separar responsabilidades y el aislamiento de dominio. Se divide en los siguientes tres niveles principales:

- **Controller:** Capa que expone los endpoints de la API REST, recibe las peticiones HTTP, extrae parámetros y delega la ejecución a nivel inferior

- **Service:** Capa con la lógica del negocio, las validaciones de la plataforma y el control trasaccional. Esta capa es la única responsable de interactuar con los repositorios y realizar las transformaciones de datos
- **Repository:** Capa de acceso a los datos, abstrayendo las operaciones de PostgreSQL mediante Spring Data JPA. Para el manejo de consultas complejas hace uso de la interfaz JpaSpecificationExecutor.

2.2 Seguridad y Autenticación

La infraestructura de seguridad está gestionada por Spring Security sin estado y eliminando el uso de sesiones en memoria.

- **Gestión de contraseñas:** Se usa BCryptPasswordEncoder configurado a través de SecurityConfig para el cifrado unidireccional de contraseñas, garantizando un almacenamiento seguro en la base de datos.
- **Autenticación JWT:** Cada petición hacia rutas protegidas es interceptada por el JwtAuthenticationFilter que extrae y valida el token JWT proporcionado y si tiene éxito, inyecta el correo electrónico del usuario en el contexto de seguridad.
- **Acceso:** El sistema restringe el acceso globalmente requiriendo autenticación para todas las operaciones (a excepción de las designadas al registro e inicio de sesión).

2.3 Flujo Request-Response

1. Cliente: El frontend (React) emite una petición HTTP adjuntando el token en la cabecera Authorization: Bearer {token};
2. Filtro: El JwtAuthenticationFilter valida el token y establece la identidad en el contexto de Spring Security.
3. Controlador: Intercepta la petición, deserializa el cuerpo en un objeto de transferencia (DTO) y llama al método correspondiente del Service
4. Servicio: Resuelve al usuario autenticado mediante el correo extraído del token, ejecuta las reglas de negocio pertinentes y consulta o modifica datos a través del Repository
5. Repositorio: Realiza las operaciones en la base de datos relacional (PostgreSQL/Supabase) mediante JPA y devuelve una Entity
6. Mapeo y Respuesta: El Service transforma la Entity en un Domain Object; posteriormente, el Controller mapea dicho dominio a un DTO de respuesta y lo serializa en formato JSON para finalizar la petición HTTP

2.4 DTOs, Domain Objects y Entities

Se usan tres representaciones distintas de los datos según la capa en la que transiten, comunicadas a través de funciones mapeadores:

- **DTOs (Data Transfer Objects):** Se utilizan en la capa Controller y contienen anotaciones de validación (ej. @NotBlank, @Size) y dictan la estructura exacta de las peticiones (Request) y respuestas (Response) HTTP expuestas al cliente
- **Domain Objects:** Clases de datos puras en Kotlin (ej. User, Publication, Photo, Interest) que actúan como la capa intermedia de transporte lógico dentro de los Services, aislando la lógica de negocio de la infraestructura
- **Entities:** Se usan solo en la capa Repository. Estas clases contienen anotaciones de JPA (@Entity, @Table, @Column) y reflejan el esquema, las restricciones y las relaciones de las tablas en PostgreSQL

2.5 Integración de servicios externos

El backend delega algunas operaciones de infraestructura a servicios externos como:

- **CloudinaryService:** Se encarga de la gestión de multimedia. Recibe archivos desde la petición, los transfiere a la nube de cloudinary y retorna una URL segura que es persistida por la entidad Photo
- **EmailService:** Emplea una interfaz JavaMailSender conectada a un servidor SMTP para enviar notificaciones por correo electrónico al propietario de una publicación cuando otro usuario marca interés en ella.
- **Geocoding y Postal Code API:** Servicio de mapas que recibe el código postal introducido en una publicación y devuelve automáticamente sus coordenadas exactas (latitud y longitud) para ubicarlas en el mapa del frontend.

3 Especificación de Requerimientos

3.1 Requerimientos Funcionales

- **RF-01. Registro de usuario**

Descripción: El sistema debe permitir que un usuario se registre proporcionando:

- Nombre y apellido
- Nombre de usuario
- Correo electrónico válido
- Código postal
- Contraseña

Criterios de aceptación: El correo y el nombre de usuario deben ser únicos en el sistema y la contraseña debe cumplir con una longitud mínima de 6 caracteres.

■ **RF-02. Autenticación de Usuario**

Descripción: El sistema debe permitir el inicio de sesión mediante correo electrónico y contraseña únicamente a usuarios registrados previamente.

Criterios de aceptación: Si las credenciales son válidas, el sistema debe retornar un token de acceso (JWT) para operaciones posteriores

■ **RF-03. Cierre de sesión**

Descripción: El sistema debe permitir a un usuario autenticado cerrar su sesión activa.

Criterios de aceptación:

- El sistema cierra la sesión eliminando el token y denegando accesos posteriores.

■ **RF-04. Consulta de información del usuario autenticado**

Descripción: El sistema debe permitir a un usuario consultar su propia información de perfil a través de un endpoint (`/users/me`).

Criterios de aceptación:

- El sistema devuelve los datos del usuario (nombre, correo, código postal).
- El sistema omite información sensible (como la contraseña cifrada).

■ **RF-05. Actualización de información del usuario autenticado.**

Descripción: El sistema debe permitir a un usuario autenticado cambiar cualquier dato (ya sea uno o varios) de su información que había registrado inicialmente. Esto a través del endpoint (`/users`).

Criterios de aceptación:

- El sistema ignora valores nulos que el usuario haya enviado.

■ **RF-06. Creación de publicación**

Descripción: El sistema debe permitir a un usuario autenticado crear una publicación de mascota especificando nombre, descripción, tipo de mascota (perro o gato), raza (opcional) y código postal

Criterios de aceptación: Las nuevas publicaciones se registran automáticamente con el estado inicial DRAFT y se les asigna al creador como propietario

■ **RF-07. Listado y filtrado de publicaciones**

Descripción: El sistema devuelve un listado de publicaciones

Criterios de aceptación:

- Se listan las publicaciones con estado ACTIVE (que tienen fotos).
- El sistema permite el filtro dinámicamente por mascota, raza y código postal
- Cada publicación incluye una foto de portada
- Un usuario no puede ver las publicaciones que el mismo publica

■ **RF-08. Consulta de detalles de publicación**

Descripción: El sistema debe mostrar el detalle completo de una publicación específica mediante su identificador

Criterios de aceptación: Si el estado de la publicación es ACTIVE, el sistema retorna los datos descriptivos junto a un arreglo de las URLs de las fotos asociadas.

■ **RF-09. Actualización de publicación**

Descripción: El sistema debe permitir la edición parcial de los detalles descriptivos de una publicación

Criterios de aceptación:

- Solo el propietario de la publicación tiene autorizado modificarla
- El sistema ignora valores nulos que el usuario haya enviado.

■ **RF-10. Eliminación de publicación**

Descripción: El sistema debe permitir borrar una publicación existente del registro.

Criterios de aceptación:

- Solo el propietario de la publicación tiene autorizado borrar su publicación
- El sistema borra permanentemente el registro

■ **RF-11. Carga de fotografías**

Descripción: El sistema debe permitir subir una o múltiples imágenes adjuntas a una publicación específica.

Criterios de aceptación:

- Solo el propietario de la publicación puede subir una foto
- Si la publicación se encuentra en estado DRAFT, tras subir la foto, el sistema cambia automáticamente a ACTIVE
- Se puede tener un máximo de hasta 8 fotografías por publicación y cada una con un límite de 5MB

■ **RF-12. Eliminación de fotografías**

Descripción: El sistema debe permitir eliminar de manera individual las fotografías asociadas a una publicación

Criterios de aceptación:

- Se verifica que la foto pertenezca a la publicación indicada
- Solo el propietario de la publicación tiene autorizado borrar su foto

■ **RF-13. Registro de interés en publicación**

Descripción: El sistema debe permitir que un usuario autenticado marque su interés en adoptar a la mascota de una publicación

Criterios de aceptación:

- Un usuario no puede estar interesado en su propia publicación

- El sistema evita registros duplicados por un mismo usuario.
- **RF-14. Consulta de usuarios interesados**
Descripción: El sistema debe devolver la lista de usuarios que han marcado interés en una publicación específica
Criterios de aceptación:
 - Solo el propietario de la publicación tiene permisos para consultar esta lista
 - La lista incluye nombre de usuario y correo de los interesados
- **RF-15. Notificaciones por correo electrónico**
Descripción: El sistema debe notificar al propietario de una publicación cuando un usuario marca interés en ella.
Criterios de aceptación:
 - El correo se envía en forma automática tras confirmar el registro de interés
 - El correo incluye los datos de contacto del usuario interesado
- **RF-16. Visualización de Mapa de Adopciones**
Descripción: El sistema debe mostrar un mapa agrupando las publicaciones activas según su ubicación geográfica.
Criterios de aceptación:
 - Se obtienen las coordenadas (latitud y longitud) consumiendo la API externa Zippopotam a partir del código postal.
 - El mapa solo muestra publicaciones en estado ACTIVE.
- **RF-17. Marcar mascota como adoptada**
Descripción: El sistema debe permitir al propietario cambiar el estado de su publicación a ADOPTED.
Criterios de aceptación:
 - Solo el propietario puede realizar esta acción.
 - La publicación debe estar previamente en estado ACTIVE.
- **RF-18. Consulta de catálogo de razas**
Descripción: El sistema debe integrar APIs externas (The Dog API y The Cat API) para autocompletar o listar las razas de mascotas disponibles.ss

3.2 Requerimientos No Funcionales

- **RNF-01. Seguridad y cifrado de contraseñas:**
Las contraseñas de los usuarios no deben almacenarse en texto plano bajo ninguna circunstancia. El sistema debe utilizar estrictamente el algoritmo de hashing unidireccional BCrypt a través de la interfaz de Spring Security.

- **RNF-02. Autenticación y Manejo de Sesiones:**

La autenticación debe ser verificada en cada petición entrante (excepto rutas públicas) mediante la validación criptográfica de un token estándar JWT (JSON Web Token) a través de un filtro interceptor

- **RNF-03. Control de acceso:**

El sistema debe garantizar que todas las operaciones de modificación, eliminación o consulta de datos sensibles (fotos, actualización de publicaciones, listado de interesados) validen criptográficamente que la identidad extraída del JWT coincida con el identificador del propietario

- **RNF-04. Validación de Datos**

Todas las peticiones HTTP entrantes deben ser validadas en la capa de Controladores antes de procesar la lógica de negocio. Se debe rechazar cualquier entrada que no cumpla el formato válido

- **RNF-05. Manejo de Imágenes en la Nube**

El almacenamiento de fotos se delega a Cloudinary. Lo que se guarda en la base de datos únicamente es la URL

4 Casos de Uso

A continuación se detallan los casos de uso principales:

- **CU-01. Registro de usuario**

Actores involucrados:

- Usuario visitante
- Sistema

Descripción breve: Permite a un usuario crear una cuenta en el sistema proporcionando información básica.

Precondiciones:

- El usuario no debe estar registrado previamente con el mismo correo.
- El sistema debe estar disponible.

Flujo principal

1. El usuario accede a la opción Registrarse.
2. El sistema muestra el formulario para registrar sus datos.
3. El usuario introduce:
 - Nombre y apellido

- Nombre de usuario
 - Correo electrónico
 - Código postal
 - Contraseña
4. El sistema valida que los campos sean correctos.
 5. El sistema verifica que el correo no exista previamente.
 6. El sistema envía un correo de confirmación
 7. El sistema almacena la contraseña de forma cifrada.
 8. El sistema registra al usuario.

Flujos alternativos

A1. Correo ya registrado

1. El sistema detecta que el correo ya existe.
2. Muestra mensaje: El correo ya está registrado.
3. El usuario debe ingresar otro correo.

A.2 Contraseña vacía

1. El sistema detecta que el usuario registró una contraseña nula.
2. Muestra mensaje: La contraseña no puede ser vacía.
3. El usuario debe ingresar una contraseña válida.

Postcondiciones

- El usuario queda registrado en el sistema.
- El usuario puede iniciar sesión.

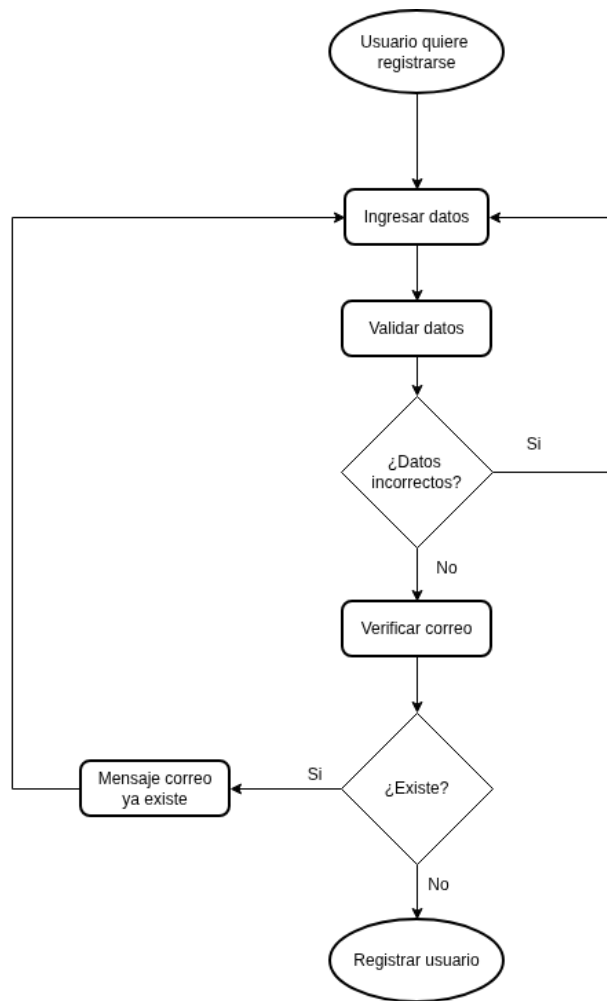


Figura 1: Diagrama de flujo registrar a un usuario

■ CU-02. Actualización de información del usuario

Actores involucrados:

- Usuario autenticado
- Sistema

Descripción: Permite a un usuario autenticado modificar su información registrada.

Precondiciones:

- El usuario debe estar autenticado con un token no nulo
- El sistema debe de estar disponible

Flujo principal:

1. El usuario autenticado con un token válido envía una petición para actualizar sus datos

2. El sistema identifica al usuario por su token y valida los datos ingresados
3. El sistema actualiza los datos que no sean nulos

Flujos alternativos

A1. Token inválido

1. El sistema no encuentra al usuario asociado al token
2. Devuelve un error 403

A2. Correo en uso

1. El sistema detecta que ese correo ya se encuentra en uso
2. Devuelve un error 403

A3. Contraseña nula

1. El sistema detecta que el usuario envió una contraseña nula
2. Devuelve un mensaje: *La contraseña no puede ser vacía*

Postcondiciones:

1. La información del usuario queda actualizada en la base de datos

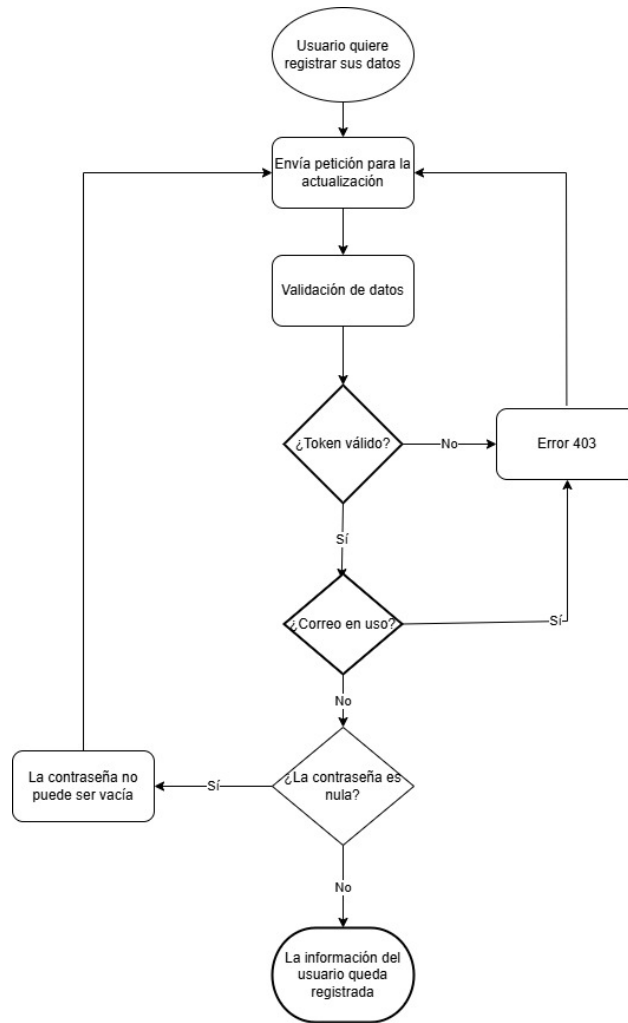


Figura 2: Diagrama de actualizar datos

■ CU-03. Crear publicacion

Actores involucrados:

- Usuario autenticado
- Sistema

Descripción: Permite a un usuario autenticado registrar una nueva publicacion de mascota. La publicacion se crea con status DRAFT y no sera visible en el listado de publicaciones hasta que se le añadan fotos.

Precondiciones:

- El usuario debe tener una sesión activa y válida en el sistema

Flujo principal:

1. El usuario solicita crear una publicación ingresando los datos de la mascota (nombre, descripción, tipo, raza opcional y código postal)
2. El sistema verifica la identidad del usuario a través de su sesión
3. El sistema vincula al usuario como propietario del nuevo registro
4. El sistema persiste la publicación asignánsole el estado inicial de DRAFT
5. El sistema confirma al usuario la creación exitosa del registro

Escenarios de error

A1. Sesión inválida

1. El sistema deniega el acceso a la funcionalidad

A2. Descripción insuficiente

1. El sistema rechaza la solicitud si la descripción es vacía

Postcondiciones:

1. La publicación existe en el sistema, pero no aparecerá en las búsquedas públicas hasta transitar a un estado ACTIVE

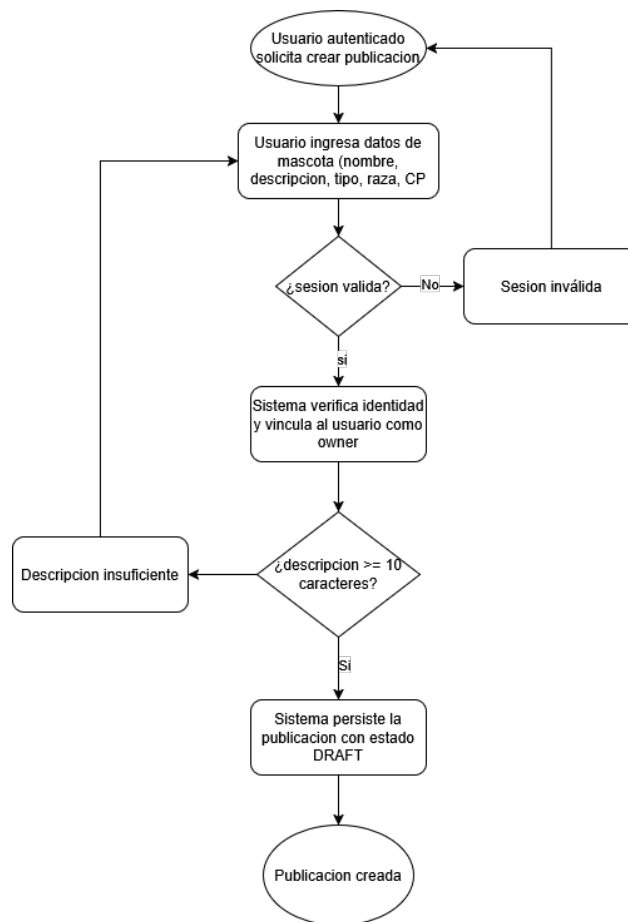


Figura 3: Diagrama de crear publicación

■ CU-04. Ver detalles de publicación

Actores involucrados:

- Usuario autenticado
- Sistema

Descripción: Muestra a cualquier usuario la información descriptiva completa de una mascota en adopción, junto con sus fotos asociadas

Precondiciones:

- El usuario debe tener una sesión activa
- La publicación debe estar en estado ACTIVE

Flujo principal:

1. El usuario solicita ver detalles de una publicación específica
2. El sistema valida la existencia de la publicación y verifica que no sea un borrador privado.
3. El sistema recopila los datos descriptivos y todas las imágenes asociadas al registro.
4. El sistema presenta la publicación al usuario

Escenarios de error

A1. Publicación no disponible

1. Si el registro no existe o su estado es DRAFT, el sistema informa al usuario que el contenido no está disponible o no existe

■ CU-05. Listar publicaciones con filtros

Actores involucrados:

- Usuario autenticado
- Sistema

Descripción: Permite ver el catálogo general de mascotas disponible soportando una aplicación de filtros, entrega resultados paginados e incluye una foto de portada por cada resultado.

Precondiciones:

- El usuario debe tener una sesión activa
- El usuario no debe ser el dueño

Flujo principal:

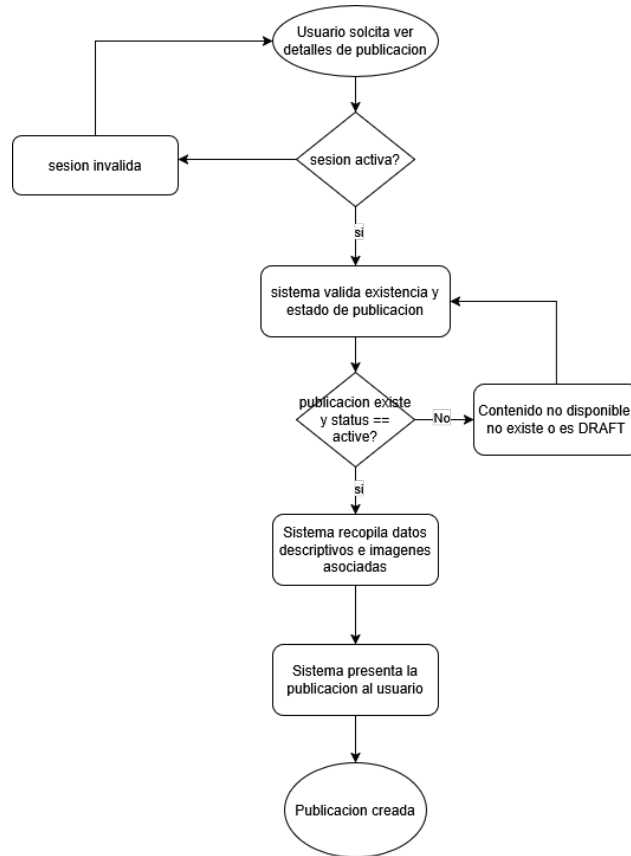


Figura 4: Diagrama de flujo registrar a un usuario

1. El usuario navega al listado de publicaciones aplicando opcionalmente parámetros de búsqueda como tipo de mascota, código postal o raza
2. El sistema procesa la consulta aplicando una restricción para devolver únicamente publicaciones con estado ACTIVE
3. El sistema aplica los filtros seleccionados ignorando mayúsculas y minúsculas en campos de texto como la raza
4. El sistema extrae la primera fotografía asociada a cada publicación para usarla como miniatura
5. El sistema devuelve y muestra la página de resultados

Flujos alternativos A1. Sin resultados

1. Si ninguna mascota cumple con los criterios de búsqueda el sistema muestra un catálogo vacío

Escenarios de error

E1. Parámetros inválidos

1. Si el usuario envía un tipo de mascota no definido por el sistema (dog/cat), se rechaza la búsqueda

Postcondiciones:

1. El usuario visualiza una lista paginada de publicaciones activas que no pertenecen a el

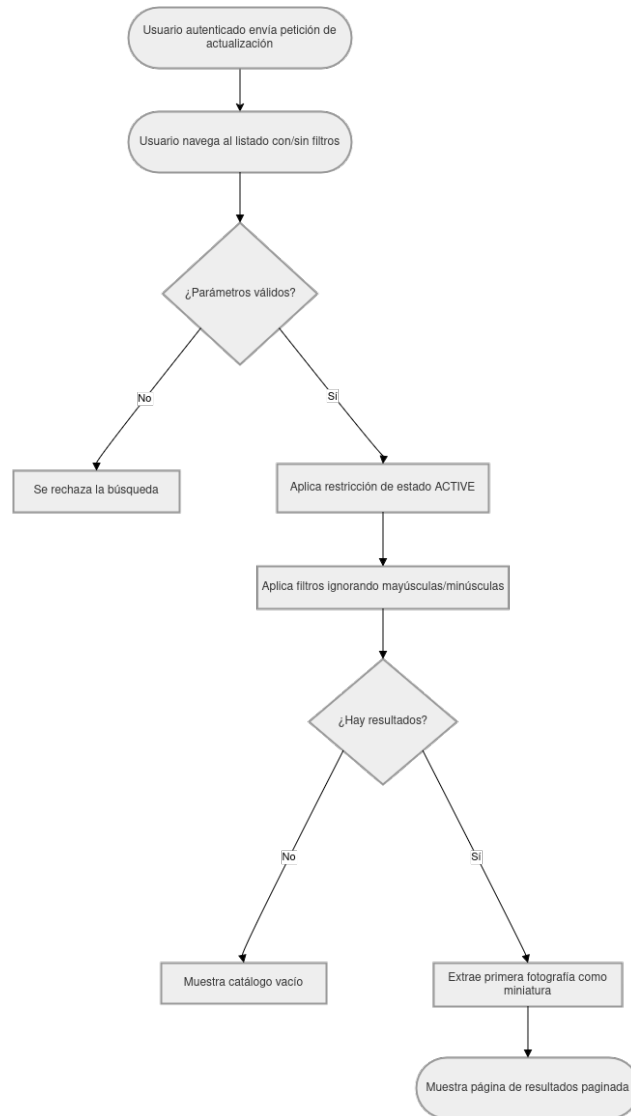


Figura 5: Diagrama de crear publicación

■ CU-06. Actualizar publicación

Actores involucrados:

- Usuario autenticado (Propietario)
- Sistema

Descripción: Permite al autor de una publicación modificar parcialmente sus datos descriptivos

Precondiciones:

- El usuario debe tener una sesión activa
- El usuario debe ser el propietario de la publicación

Flujo principal:

1. El usuario envía los nuevos valores de los campos que desea modificar
2. El sistema verifica la existencia de la publicación
3. El sistema valida que el solicitante sea el propietario de la publicación
4. El sistema actualiza únicamente los campos enviados conservando los demás valores
5. El sistema guarda la información actualizada
6. El sistema confirma la actualización exitosa

Escenarios de error

E1. Violación de propiedad

1. Si el usuario intenta editar una publicación ajena, el sistema bloquea la acción por falta de permisos

E2. Información insuficiente

1. Si la nueva descripción es vacía el sistema rechaza la actualización

Postcondiciones:

1. La publicación queda actualizada en la base de datos

■ **CU-07. Eliminar publicación**

Actores involucrados:

- Usuario autenticado (Propietario)
- Sistema

Descripción: Permite a un dueño retirar permanentemente su publicación

Precondiciones:

- El usuario debe tener una sesión activa
- El usuario debe ser propietario de la publicación

Flujo principal:

1. El usuario solicita la eliminación de su publicación

2. El sistema autentica la solicitud
3. El sistema valida que el usuario sea el propietario del registro
4. El sistema elimina la publicación y toda la información asociada
5. El sistema confirma la eliminación exitosa

Escenarios de error

E1. Violación de propiedad

1. Si el usuario intenta eliminar una publicación ajena el sistema deniega la operación

Postcondiciones:

1. La publicación y sus fotografías se eliminan permanentemente sin posibilidad de recuperación

■ **CU-08. Subir fotos a publicación**

Actores involucrados:

- Usuario autenticado (Propietario)
- Sistema

Descripción: Permite al creador adjuntar archivos de imagen a su publicación. Además, si la publicación no tenía fotos, la publicación pasa a ser pública

Precondiciones:

- El usuario debe tener una sesión activa
- El usuario debe ser propietario de la publicación
- Se debe proporcionar al menos una imagen

Flujo principal:

1. El usuario selecciona y envía una o varias fotografías
2. El sistema carga las imágenes en la nube
3. El sistema obtiene las URLs generadas
4. El sistema vincula las URLs a la publicación
5. Si la publicación estaba en estado DRAFT, el sistema la cambia a ACTIVE
6. El sistema confirma la carga exitosa y muestra las imágenes que se adjuntaron

Escenarios de error

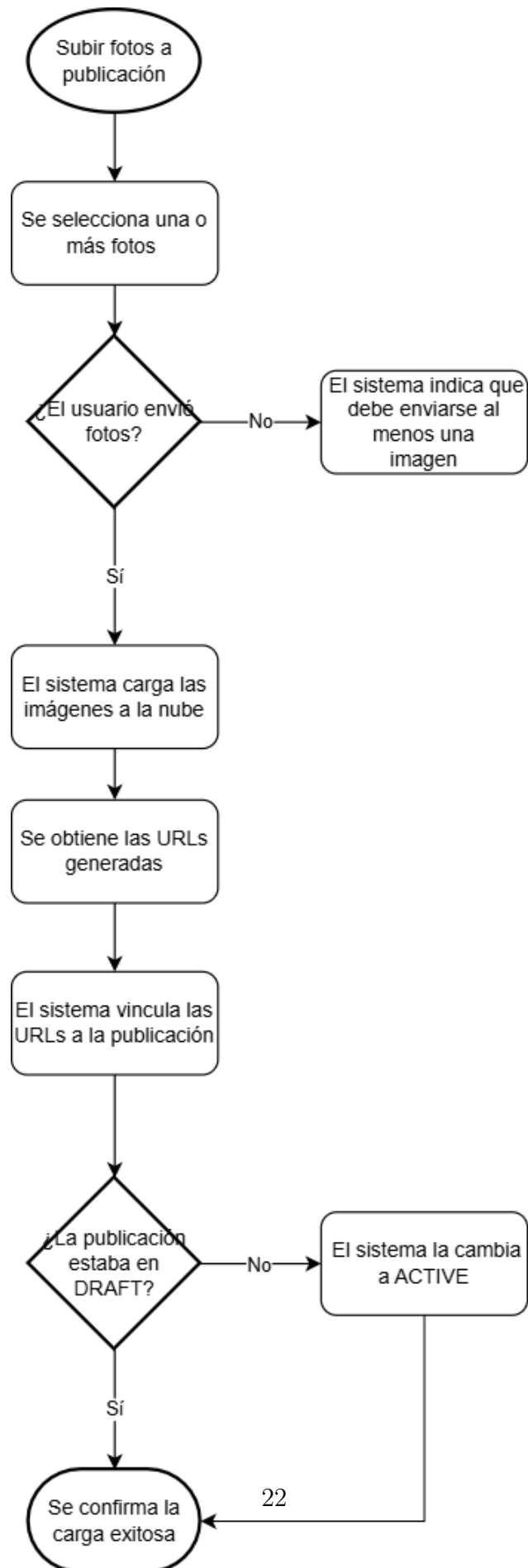
E1. Petición vacía

1. Si el usuario no envía imágenes el sistema indica que debe enviarse al menos una imagen
2. Si el usuario quiere subir más de 8 fotos, el sistema no se lo permite

3. Si el usuario quiere subir una imagen con un tamaño mayor a 5MB el sistema no se lo permite

Postcondiciones:

1. Las imágenes quedan asociadas a la publicación
2. La publicación puede cambiar de estado DRAFT a estado ACTIVE



■ CU-09. Eliminar foto de publicación

Actores involucrados:

- Usuario autenticado (Propietario)
- Sistema

Descripción: Permite al propietario quitar una imagen específica de las fotos de su publicación.

Precondiciones:

- El usuario debe tener una sesión activa
- El usuario debe ser propietario de la publicación
- La imagen debe pertenecer a la publicación indicada

Flujo principal:

1. El usuario solicita eliminar una fotografía específica
2. El sistema verifica que la imagen exista
3. El sistema valida que la fotografía pertenezca a la publicación indicada
4. El sistema elimina el registro de la fotografía
5. El sistema confirma la eliminación exitosa

Escenarios de error

E1. Inconsistencia referencial

1. Si la imagen no corresponde a la publicación indicada el sistema rechaza la solicitud y notifica el error

Postcondiciones:

1. La fotografía deja de estar asociada a la publicación

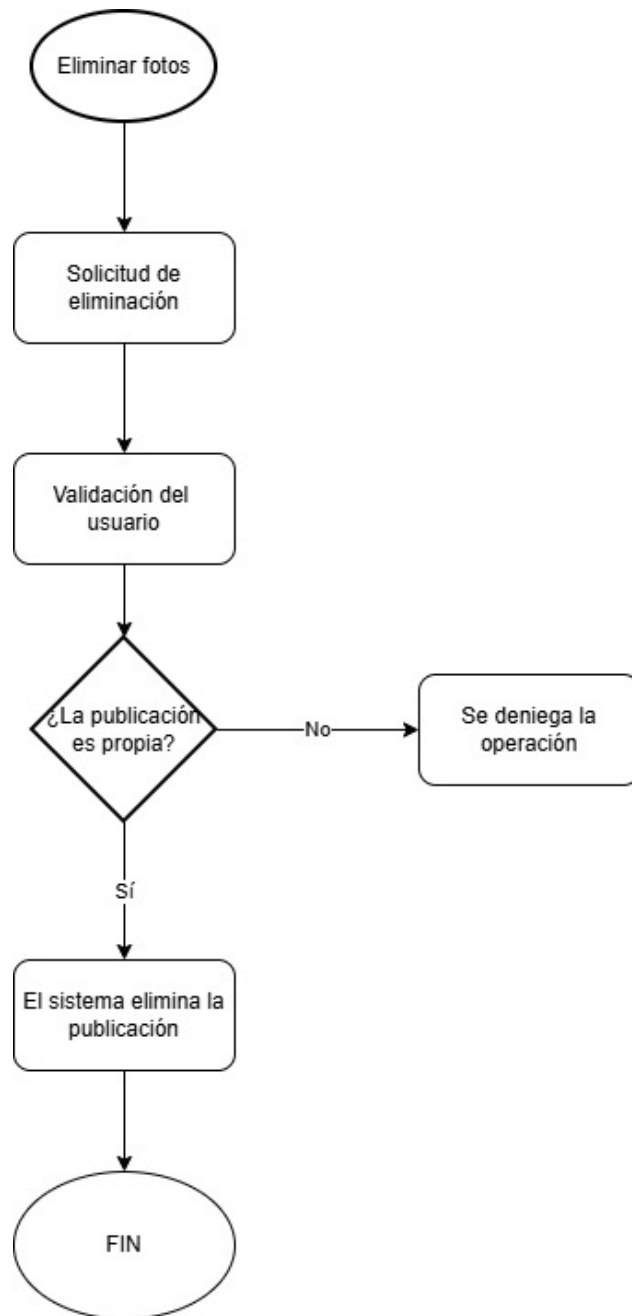


Figura 7: Diagrama de eliminar foto

■ CU-10. Marcar interés en publicación

Actores involucrados:

- Usuario autenticado (Interesado)
- Sistema

Descripción: Permite a un usuario expresar interés en adoptar una mascota y notifica

automáticamente al propietario.

Precondiciones:

- El usuario debe tener una sesión activa
- El usuario no debe ser el creador de la publicación
- No debe existir un interés previo registrado

Flujo principal:

1. El usuario confirma su interés en adoptar a la mascota
2. El sistema valida que el usuario no sea el dueño de la publicación
3. El sistema verifica que antes no haya tenido interés para la misma mascota
4. El sistema registra el interés
5. El sistema recopila la información de contacto del interesado
6. El sistema envía una notificación por correo al propietario
7. El sistema informa al usuario que el interés fue registrado exitosamente

Escenarios de error

E1. Adopción propia

1. El usuario intenta marcar interés en su propia mascota
2. El sistema rechaza la solicitud

E2. Interés duplicado

1. Si el usuario ya había registrado interés previamente el sistema evita la duplicidad y notifica al usuario

Postcondiciones:

1. El interés queda registrado en el sistema
2. El propietario recibe una notificación por correo

■ **CU-11. Ver usuarios interesados en publicación**

Actores involucrados:

- Usuario autenticado (Propietario)
- Sistema

Descripción: Permite al propietario ver la lista de usuarios interesados en adoptar a su mascota.

Precondiciones:

- El usuario debe tener una sesión activa

- El usuario debe ser el propietario de la publicación

Flujo principal:

1. El usuario solicita visualizar los interesados de una publicación
2. El sistema valida que el solicitante sea el propietario de la publicación
3. El sistema recupera todos los registros de interés que tenga
4. El sistema obtiene el nombre y correo electrónico de cada interesado
5. El sistema muestra la lista completa al propietario

Flujos alternativos**A1. Lista vacía**

1. Si no existen interesados registrados para la publicación el sistema muestra una vista vacía

Escenarios de error**E1. Violación de propiedad**

1. Si alguien distinto al propietario intenta consultar la lista, el sistema deniega el acceso

Postcondiciones:

1. El propietario ve la información de contacto de los interesados

■ CU-12. Visualizar mapa de publicaciones**Actores involucrados:**

- Usuario autenticado
- Sistema
- API Externa (Zippopotam)

Descripción: Permite a un usuario ver las publicaciones activas agrupadas por ubicación geográfica en un mapa, utilizando el código postal.

Precondiciones:

- El usuario debe tener una sesión activa.

Flujo principal:

1. El usuario accede a la sección del mapa aplicando filtros opcionales (tipo, raza, código postal).
2. El sistema filtra las publicaciones asegurando que estén en estado ACTIVE.
3. El sistema agrupa las publicaciones por código postal.
4. El sistema consulta la base de datos local o la API Zippopotam para obtener latitud y longitud.

5. El sistema devuelve las coordenadas y la primera foto de cada publicación para ser renderizada en el mapa.

Escenarios de error

E1. Código postal inválido

1. Si la API Zippopotam no encuentra el código postal, el sistema omite esa ubicación devolviendo un error controlado o un arreglo vacío.

Postcondiciones:

1. El usuario visualiza los clústeres de mascotas en su ubicación correspondiente.

■ **CU-13. Marcar publicación como adoptada**

Actores involucrados:

- Usuario autenticado (Propietario)
- Sistema

Descripción: Permite al propietario cambiar el estado de su publicación a ADOPTED indicando que la mascota ha encontrado un hogar.

Precondiciones:

- El usuario debe tener una sesión activa y ser el propietario de la publicación.
- La publicación debe encontrarse en estado ACTIVE.

Flujo principal:

1. El usuario solicita marcar su publicación como adoptada.
2. El sistema valida que el usuario sea el creador del registro.
3. El sistema verifica que la publicación tenga estado ACTIVE.
4. El sistema actualiza el estado de la publicación a ADOPTED en la base de datos.
5. El sistema confirma la adopción exitosa.

Escenarios de error

E1. Estado previo incorrecto

1. Si la publicación se encuentra en estado DRAFT, el sistema lanza una excepción impidiendo la operación.

Postcondiciones:

1. La publicación deja de ser visible en el listado general de adopciones.

■ **CU-14. Consultar catálogo de razas desde API Externa**

Actores involucrados:

- Usuario autenticado

- Sistema
- API Externa (The Dog API / The Cat API)

Descripción: Recupera una lista estandarizada de razas para facilitar el registro de una mascota.

Precondiciones:

- El usuario debe tener una sesión activa.

Flujo principal:

1. El usuario selecciona el tipo de mascota (DOG o CAT) en el formulario.
2. El sistema detecta el tipo y realiza una petición a la API externa correspondiente.
3. El sistema recibe el catálogo de razas con detalles (origen, temperamento).
4. El sistema devuelve la lista formateada al usuario para su selección.

Flujos alternativos

A1. Caída de API Externa

1. Si la API externa no responde, el sistema consulta el caché de la base de datos local y devuelve las razas previamente guardadas.

5 Diseño de Datos

5.1 Modelo Conceptual de Datos

El sistema tiene cuatro entidades lógicas principales que participan en el flujo para la adopción de mascotas, las cuales son:

- **User:** Es la entidad principal del sistema y almacena información de identidad y credenciales de acceso
- **Publication:** Entidad que representa el anuncio de una mascota en adopción
- **Photo:** Entidad que representa los multimedia subidas a una publicación
- **Interest:** Entidad transaccional que modela el interés que un usuario tiene en adoptar alguna mascota.

5.2 Schema

- Para el usuario:
 - id.user: Identificador unico del usuario. PK de tipo BIGSERIAL
 - username, first_name, last_name: Identidad del usuario. Son VARCHAR con restricciones UNIQUE (solo username) y NOT NULL

- email: Identificador para iniciar sesión. Es de tipo VARCHAR con restricciones UNIQUE y NOT NULL
 - zip_code: Código para la futura localización. Es de tipo VARCHAR y NOT NULL
 - password: Credencial para que el usuario pueda acceder (se almacena de forma segura usando un cifrado hash). Es de tipo VARCHAR y NOT NULL
 - is_verified: Booleano que indica si el correo ya fue verificado
- Para Publication:
- id: Identificador único de la publicación Es una PK de tipo BIGSERIAL
 - owner_id: Identificador del usuario creador (Propietario). Es de tipo BIGINT con restricciones NOT NULL y FK a users.id
 - pet_name: Es el nombre de la mascota en adopción. Es de tipo VARCHAR con restricción NOT NULL
 - description: Descripción detallada de la mascota. Es de tipo VARCHAR(1000) y con restricciones NOT NULL, min length 10
 - type: Tipo de mascota (DOG, CAT). Es de tipo VARCHAR (Enum) con restricciones NOT NULL
 - breed: Es la raza de la mascota. Es de tipo VARCHAR y puede ser NULL
 - zip_code: Código postal donde se ubica la mascota. Es de tipo VARCHAR y con restricciones NOT NULL
 - status: Estado de visibilidad (DRAFT, ACTIVE). Es de tipo VARCHAR (Enum) con restricciones NOT NULL
 - breed_info_id: Es una llave FK BIGINT que vincula la publicación con la información de la raza en la API
- Para Photo:
- id: Identificador único del registro multimedia. Es PK de tipo BIGSERIAL
 - : publication_id: Identificador de la publicación asociada. Es de tipo BIGINT con restricciones NOT NULL y FK a publications.id
 - url: Enlace seguro (HTTPS) proveído por Cloudinary. Es de tipo VARCHAR y con restricciones NOT NULL
 - public_id: Permite al sistema conectarse con Cloudinary para eliminar imágenes, de tipo VARCHAR y no nulo
- Para Interest:
- id: Identificador único del registro de interés. Es PK de tipo BIGSERIAL
 - publication_id: ID de la publicación que recibe el interés. Es de tipo BIGINT con restricciones NOT NULL y FK a publications.id

- `interested_user_id`: ID del usuario que manifiesta el interés. Es de tipo `BIGINT` con restricciones `NOT NULL` y `FK` a `users.id`
- Para `Verification Token`:
 - `id`: Identificador único. PK de tipo `BIGSERIAL`.
 - `token`: Cadena única generada para verificar el correo. Es de tipo `VARCHAR` y `UNIQUE`.
 - `user_id`: Identificador del usuario al que pertenece. `FK` a `users.id` con restricción `ON DELETE CASCADE`.
 - `expiry_date`: Fecha de expiración del token. Es `TIMESTAMP`.
- Para `Breed Info`:
 - `id`: Identificador único. PK de tipo `BIGSERIAL`.
 - `type`: Tipo de mascota (`DOG`, `CAT`).
 - `breed_name`: Nombre estandarizado de la raza.
 - `temperament`, `origin`, `life_span`: Información descriptiva obtenida de las APIs externas.
 - `description`: Es la que se obtiene de la API de tipo `VARCHAR 2000`
- Para `Postal Code Location`:
 - `zip_code`: Código postal. Es PK de tipo `VARCHAR`.
 - `lat`, `lng`: Coordenadas geográficas (latitud y longitud) de tipo `DOUBLE PRECISION`.

6 Sección Técnica

6.1 Tecnologías Utilizadas

- **Lenguaje de programación:** Kotlin. Elegido por su sintaxis, seguridad contra nulos (`null-safety`) y excelente interoperabilidad en backend.
- **Framework:** Spring Boot. Utilizado porque facilita la configuración del servidor API REST y agiliza la inyección de dependencias.
- **Seguridad y Autenticación:** Spring Security apoyado por la librería `JJWT` usada para la generación, firma y validación de los tokens JWT
- **Spring Data JPA:** Para la abstracción ORM usada en los repositorios para interactuar con la base de datos. Permite ejecutar operaciones CRUD
- **Base de Datos:** PostgreSQL. Se seleccionó por ser un motor relacional robusto que asegura la integridad referencial y un alto rendimiento. Además, se usa Supabase para alojar la base de datos en la nube.

- **Frontend:** React y React Router DOM. Elegidos para construir una interfaz de usuario interactiva usando componentes y gestionar la navegación entre vistas públicas y protegidas.
- **Comunicación HTTP:** Fetch API. Nativo de JavaScript utilizado para usar la API REST del backend.
- **Herramientas:** Maven para gestionar dependencias del proyecto y Postman para la validación y pruebas de extremo a extremo (E2E) de los endpoints. También se usa Cloudinary para recepción de archivos multimedia en la nube, JavaMailSender para el envío automático de correos electrónicos mediante SMTP, Zippopotam para la obtención de coordenadas geográficas a partir de códigos postales, y The Dog API / The Cat API el registro y autocompletado de razas.

6.2 Flujo de Autenticación

- **Inicio de sesión y Autenticación:** El usuario envía sus credenciales desde el frontend y el backend valida la contraseña utilizando BCryptPasswordEncoder y si es correcta genera y responde con un token JWT. El sistema no almacena sesiones de memoria
- **Almacenamiento temporal:** El frontend captura el token y lo persiste en el sessionStorage. No se almacenan sesiones ni tokens en la base de datos del backend.
- **Protección de rutas y CORS:** El sistema valida constantemente el acceso, a nivel de backend toda petición hacia rutas privadas es interceptada por un JwtAuthenticationFilter. Además las políticas CORS restringen las peticiones para que solo sean aceptadas desde http://localhost:3000.
- **Peticiones protegidas e Identidad:** En las consultas de información o actualizaciones el token se envía dentro del header Authorization. Si es validado el filtro inyecta el correo del usuario en authentication.name, que se usa internamente por la capa de servicios para resolver la identidad del solicitante y validar la propiedad ownership
- **Manejo de errores:** Si las credenciales son incorrectas, o si el token expira, o está mal formado, la solicitud es delegada a JwtAuthenticationEntryPoint, el error regresa HTTP 403 Forbidden y un cuerpo JSON que el frontend interpreta para redirigir al inicio de sesión o mostrar alertas visuales al usuario.

7 Actores de sistema

- Actores primarios
 - Usuario visitante: Esta en la plataforma sin haber iniciado sesión
 - Usuario autenticado: Cuando ya se registró e inició sesión
 - Propietario: Cuando el usuario crea una publicación y solo el tiene los permisos para editarla, eliminarla o ver la lista de interesados

- Interesado: Usuario autenticado que muestra interes en la publicacion de una mascota publicada
- Actores secundarios
 - Sistema de correo electrónico
 - Servicio de almacenamieto de multimedia (cloudinary)

8 Definiciones relevantes

Término	Definición
Usuario registrado	Persona con cuenta creada en el sistema
Propietario	Usuario que crea una publicación y tiene derechos exclusivos sobre ella
API externa	Servicio de terceros (ej. mapas, correo)
Borrador (DRAFT)	Publicacion recién creada que no cuenta con una imagen todavía, no es publica
Publicacion Activa (ACTIVE)	Publicacion que ya cuenta con una foto como mínimo y por lo tanto ya es visible
Interesado	Usuario que presiona un botón para notificar al propietario de la publicacion que tiene intención de adotar